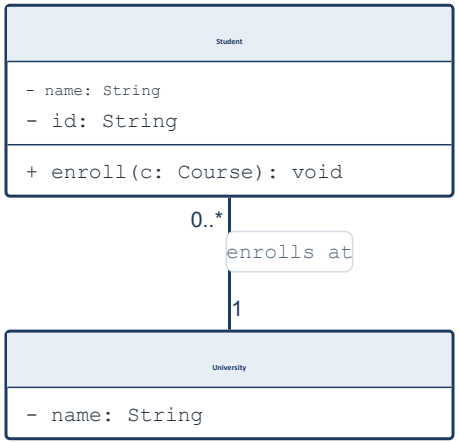
# UML Class Diagrams

The blueprint language of software structure



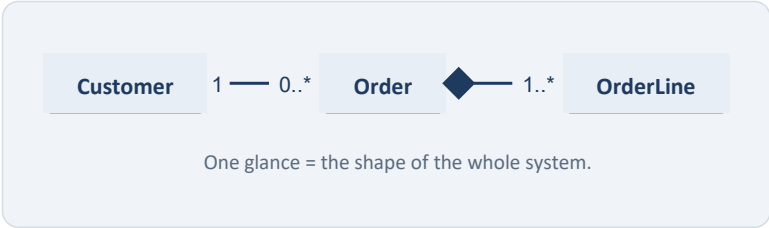
WHY MODEL AT ALL?

# Would you build a house without a blueprint?

Nobody pours concrete from a verbal description. Architects draw Architects draw blueprints: precise, reviewable, shareable, and and everyone argues about the drawing, not the building.

Software has the same problem, plus one more: you cannot see it at see it at all. A 50,000-line system is invisible without a map.

**A class diagram is that map.** It shows what exists and how everything connects before a single line of code is written.

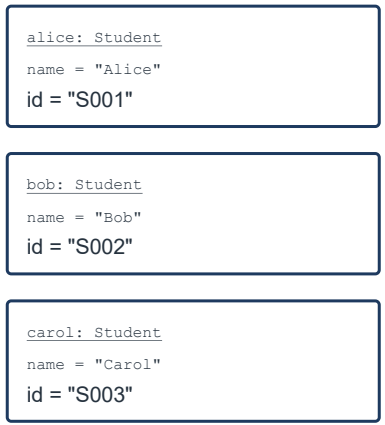


# One language, many diagrams, where we are

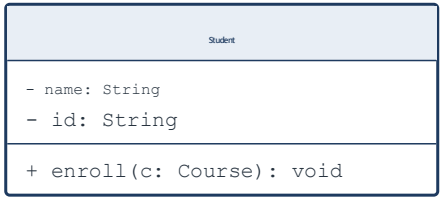
| STRUCTURAL: what the system IS | BEHAVIORAL, what the system DOES |
| --- | --- |
| **Class diagram** TODAY | Use case diagram |
| Object diagram | Activity diagram |
| Component diagram | Sequence diagram |
| Deployment diagram | Communication diagram |
| Package diagram | Interaction overview |
| Composite structure | Timing diagram |
| Profile | State machine diagram |

The class diagram is the most used diagram in practice, the backbone of object-oriented design.

# First the object. Then the class.




one class describes them all

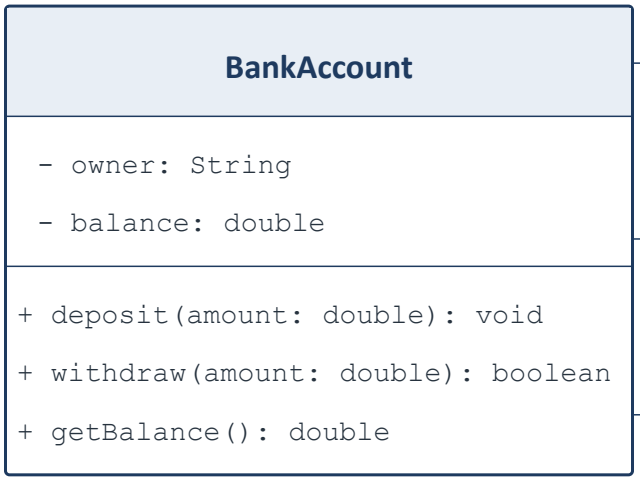



**Object** = state + behavior + identity

**Class** = the blueprint that stamps out identical-shaped objects objects

You are an object. `Student` is your class.

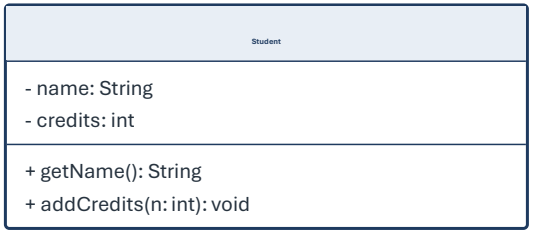
## The class box: three compartments, one rectangle




1. **Name:** bold, centered. Names the concept, not a variable: BankAccount, not acc1.
2. **Attributes:** the data each object carries. Grammar: visibility name : type
3. **Operations:** the behavior you can ask for. Grammar: visibility name(params) : returnType

Everything else in this lecture is a variation on this one rectangle.

# The same thing, in Java




```
public class Student {
private String name;    // private → '-'
private int credits;    // private → '-'

public String getName() {  // public → '+'
return name;
}
public void addCredits(int n) {
credits += n;
}
}
```

UML: type **AFTER** name (name: String) · Java: type **BEFORE** name (String name), same information, different order.

If it has a box, it has a class in code. Guaranteed.

# Attributes: the full grammar

**visibility name : type** [multiplicity] = default {properties}

| | |
|---|---|
| - balance: double | the everyday form |
| - id: String = "S000" | **default value**: assigned at creation |
| - grades: Grade [*] | attribute multiplicity: this attribute holds a **LIST** of Grade objects |
| / age: int | **DERIVED**: computed from birthDate, never stored, always consistent |
| + nickname: String = "" | public with empty default |

Types can be **primitives or classes**

Derived ( / ) = a formula, not a field: saves storage and bugs

# Four visibility levels

| Symbol | UML name | Java equivalent | Who can see it |
|---|---|---|---|
| + | public | `public` | anywhere |
| − | private | `private` | only inside this class |
| # | protected | `protected` | this class + subclasses + same package |
| ~ | package | `(no keyword)` | classes in the same package only |

| | | | |
|---|---|---|---|
| + = your front door | − = your diary | # = a family heirloom (shared with relatives = subclasses) | ~ = the corridor of your apartment building |

Attributes default to: **(private)**. **Encapsulation:** objects guard their own state — nobody reaches in and sets balance = −9999.

# Operations: behavior with a contract

**visibility name**(parameter : type, ...) : **returnType**

| | |
|---|---|
| + deposit(amount: double): void | void = a **COMMAND**: changes state, returns nothing |
| + getBalance(): double | a **QUERY**: changes nothing, returns a value |
| + transfer(to: Account, amount: double): boolean | parameters use the same name : type grammar; boolean = success/failure |
| # applyMonthlyFee(): void | **protected helper:** subclasses can call and override it |

**Queries vs commands:** a distinction that keeps APIs honest. A getBalance() that secretly changes the balance is a design bug.

## Two marks that make diagrams exact

### Static: underline it




Underlined = one value shared by **ALL** objects of the class. Belongs to the class, not to any object. Java echo:

```
static int idCounter
```

### Abstract: italicize it




Abstract operations have **NO** body; every concrete subclass must implement charge(). Payment itself can never be instantiated. Java echo:

```
abstract class Payment { abstract boolean charge(); }
```

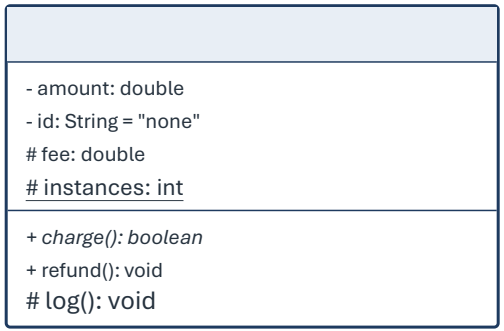
### Interface: stereotype it




A near-pure contract: zero state, signatures only. Realized by classes, next section. Real world: a USB port is a contract; anything shaped right just works. Java echo:

```
interface Payment
```

# Read this box




**Q1** Subclasses can directly touch which members?

**Q2** What does the underline on instances mean?

**Q3** Why is charge() italic, and what must subclasses do?

**Q4** Who is allowed to call refund()?

## Relationships

Six ways classes connect, and the arrows that spell the difference.

THE SIX LINE NOTATIONS



# Classes alone are just nouns

A phone book of classes with no connections is not a design. **The lines decide what holds a reference to what,** in code, that means fields, and at runtime, it means how objects find each other and collaborate. Get the lines right and behavior almost writes itself.

| | |
|---|---|
| SOCIAL NETWORK | the friend edges between Person objects: a graph of associations |
| A CAR | one composite object: thousands of parts whose lifecycles are bound inside the whole |
| A FAMILY TREE | Generalization: a Student **IS-A** Person, inherits the family traits |

### Coming up

01 **the plain line:** association

02 **the numbers:** multiplicity

03 **two diamonds:** aggregation vs composition

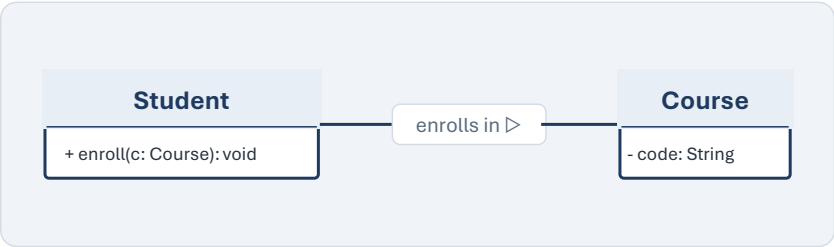
04 **two triangles:** generalization & realization

05 **the dashed arrow:** dependency

You must recognize **all six on sight**.

# Association: the plain solid line

A structural has-a / knows-a: one object holds a reference to another, long term. **If it is a field, it is a line.**




**Read it aloud:** Student enrolls in Course. Every association is a sentence you can say out loud: if you cannot say it, it is probably wrong.

```
public class Student {
private List<Course> courses;   // ← the association, as a field

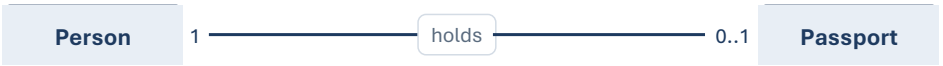
public void enroll(Course c) {
courses.add(c);
}
}
```

**The class holds a reference ⇒ draw a line.** Bidirectional so far, we add direction and numbers next.

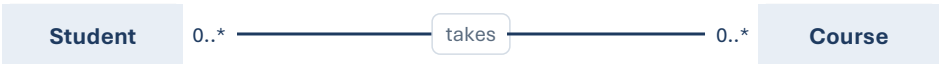
# Multiplicity: how many, at which end

The number sits at the **END** of the line, and it counts objects of the class it sits next to.

| | | |
|---|---|---|
| 1 | → | exactly one |
| 0..1 | → | zero or one (optional) |
| * = 0..* | → | zero or more |
| 1..* | → | one or more |
| m..n | → | between m and n |




each person holds at most one passport; every passport has exactly one holder




many-to-many: students take courses, courses hold students

**THE EXAM TRAP:** the 0..* near Course answers how many **COURSES** one student has. The number at an end **ALWAYS** counts that end's class.

# The multiplicities you will actually use

### Customer 1 — 0..* Order

Each order belongs to exactly one customer; a customer may have many orders or none yet.

```
private Customer customer;  (in Order)
```

### Person 1 — 0..1 Passport

Optional single reference: the passport may not exist yet; null is expected and fine.

```
private Passport passport;  // may be null
```

### Student 0..* — 0..* Course

Many-to-many. In code usually a collection on one side, or an association class (later).

```
private List<Course> courses;
```

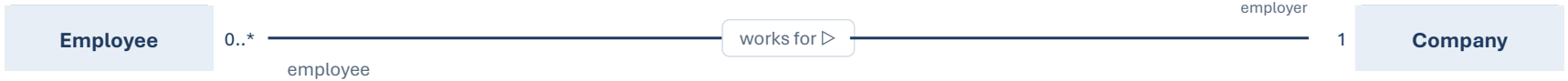
### Team 1 — 1..* Player

At least one: a team is never empty. Creation logic must guarantee the first player.

```
private List<Player> players;  + guard in constructor
```

Every multiplicity is a concrete code decision: **one reference** · **an optional reference** · **a collection**.

# A line can carry more than a number




### Association name

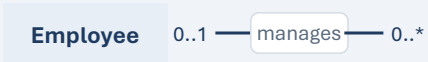
a verb phrase on the line; the small solid arrowhead ▷ fixes the reading direction

### Role names

what each end is called in this relationship: employer, employee

### Navigation

open arrow = who can reach whom. No arrow = unspecified or both directions




an association from a class back to itself, **your boss is an Employee too.**

# Read the model

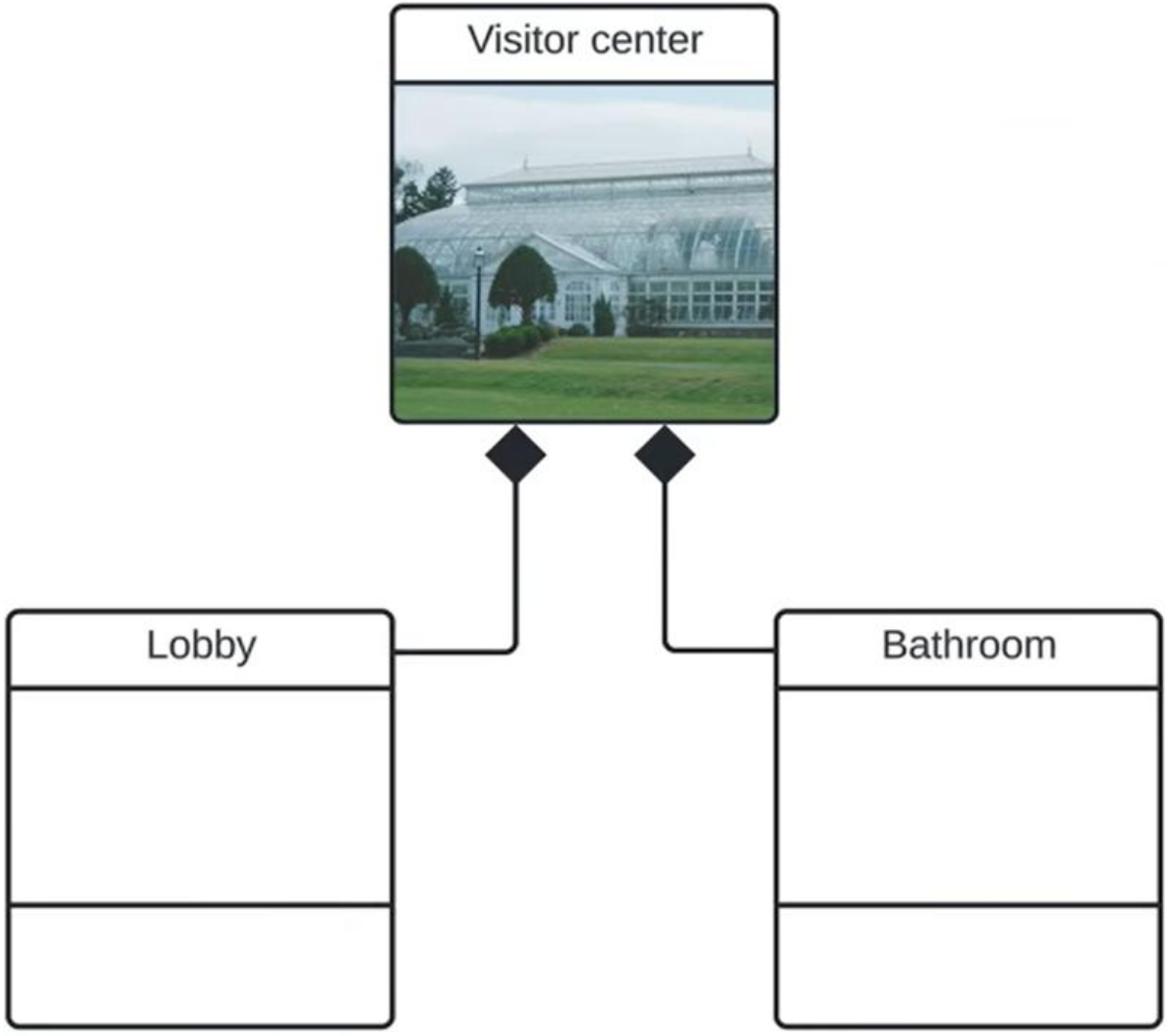



Q1 Can a doctor be a primary physician for zero patients?

Q2 Can a patient have two primary physicians?

Q3 True or false: a ward with zero nurses is valid in this model.

Q4 What field does class Nurse have, Ward ward or List<Ward> wards?


Visitor center
Lobby
Bathroom

# Aggregation: the hollow diamond ◇

A whole–part where the part can outlive the whole. **Diamond sits at the WHOLE end.**




**Disband the team → the players live on.** Free agents sign elsewhere; players existed before the club and after it.

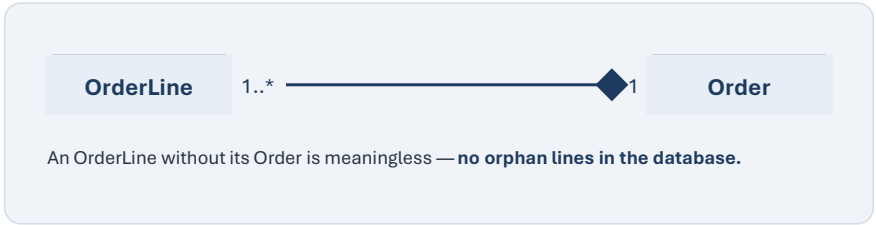
```
public class Team {
private List<Player> players;  // references to EXISTING objects

public void sign(Player p) {
players.add(p);          // the team does not create p
}
}
```

> “Precise semantics of shared aggregation varies by application area and modeler.” **Translation:** the hollow diamond means what your team agrees it means, the lifecycle test is the practical convention.

# Composition, the filled diamond ◆

A whole–part where the part's lifecycle is bound to the whole. **Delete the whole → the parts die.**







rooms do not survive demolition

```
public class Order {
private List<OrderLine> lines = new ArrayList<>();

public OrderLine addLine(Product p, int qty) {
OrderLine l = new OrderLine(p, qty);  // born INSIDE the whole
lines.add(l);
return l;
}
}
```

**UML rule:** a part belongs to at most **ONE** composite

Parts typically created by the whole

Database echo: ON DELETE CASCADE

# ◇ vs ◆: one question settles it

**THE DELETION TEST:** if the whole is destroyed, do the parts die?

## Aggregation

parts independent, they outlive the whole

a part may be reused by another whole

parts arrive from outside (constructor / setter params)

typical pairs: Team–Player · Playlist–Song

code shape: references passed in

## Composition

parts bound to the whole's lifecycle

a part belongs to exactly one whole

parts created inside the whole

typical pairs: Order–OrderLine · House–Room

code shape: object minted internally, cascade delete

**Gray zone: Car–Engine?** The engine can be pulled out and resold, yet it is built for the car. Real designs are ambiguous: it is a **MODELING JUDGMENT**. Choose, and be ready to defend the choice in the design review.

## ◇ or ◆?

**Band, Members**

members quit and the band tours on; break up, and everyone joins new bands

**Book, Chapters**

delete the book from the catalog, the chapters have no meaning alone

**Playlist, Songs**

delete the playlist, the songs live on in the library

**Order, OrderLines**

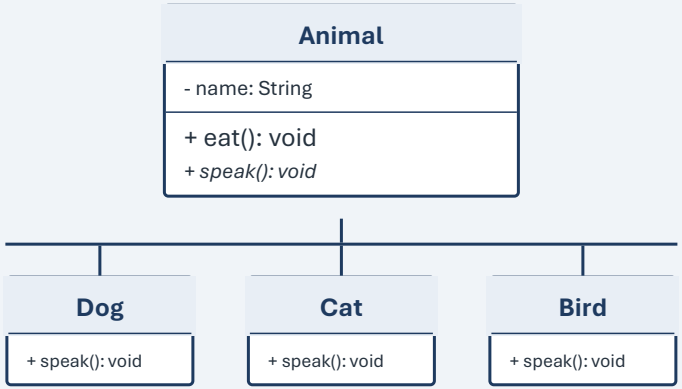
delete the order, its lines cannot exist alone; orphan rows are forbidden

**Car, Engine**

scrap the car, the engine can be pulled out and resold

# Generalization: is-a inheritance

Solid line + hollow triangle pointing at the **PARENT**.



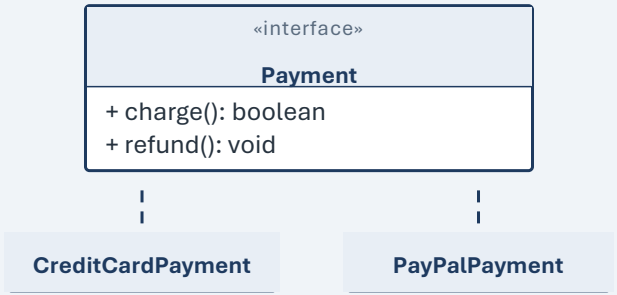

```
public class Dog extends Animal {  // extends = generalization
@Override
public void speak() {
System.out.println("Woof");
}
}
```

Inheritance is the **STRONGEST coupling** in OO. Use it only for true is-a, anywhere code expects an Animal, a Dog must fit without surprises (**Liskov Substitution Principle,** full story in a later lecture).

**Gang of Four, Design Patterns (1994):** “Favor object composition over class inheritance.”

# Realization: implementing a contract

DASHED line + hollow triangle pointing at the **interface**.




```
public class CreditCardPayment implements Payment {
@Override
public boolean charge() {
        // card gateway call
return true;
}
@Override
public void refund() {
        // reverse the transaction
}
}
```

The interface says **WHAT**. The classes say **HOW**.

Alternative notation: the lollipop, a small circle on a stick at the class edge

# Dependency: uses-a, the weakest link

DASHED line + OPEN arrow → pointing at the used class. **No field, just a fleeting reference.**




**No field** of type Database anywhere in ReportGenerator, just borrowed inside one method.

```
public class ReportGenerator {
    // NOTE: no 'private Database db' field here!

public Report build(Database db) {  // 1. method parameter
var rows = db.fetchSales();     // 2. used locally
return new Report(rows);
}
}
```

Every dependency is a **recompile risk**: change Database and build() may break. Architects count arrows, fewest dependencies wins. Common labels: «use» «create» «call».

# Six relationships at a glance

| Relationship | Line | Marker | Meaning | In Java |
|---|---|---|---|---|
| **Association** | solid | (plain line) | has-a, long-term | a field |
| **Aggregation** | solid | at the whole | has-a; part survives | field, parts from outside |
| **Composition** | solid | at the whole | has-a; part dies with whole | field, created inside |
| **Generalization** | solid | hollow triangle at parent | is-a | extends |
| **Realization** | dashed | hollow triangle at contract | fulfills a contract | implements |
| **Dependency** | dashed | open arrow → | uses-a, briefly | parameter / local |

**Solid** owns it. **Dashed** borrows it. **Diamonds** are parts. **Triangles** are contracts.

# Name that arrow

```
class SavingsAccount extends Account { ... }
```

1 → **which UML relationship?**

```
// in Playlist; tracks exist on their own
private List<Track> tracks;
```

2 → **which UML relationship?**

```
// in Printer; no field of type Invoice

void print(Invoice i) { ... }
```

3 → **which UML relationship?**

```
class SqlPayment implements PaymentGateway { ... }
```

4 → **which UML relationship?**

```
// OrderLine created inside Order

order.addLine("Espresso", 2);
```

5 → **which UML relationship?**

03

# From words to diagrams

Turning a paragraph of English into a model a developer can build from.

# Nouns become classes. Verbs become lines.

1. **NOUNS** → candidate classes (and attributes)
2. Filter: property-nouns become attributes, *"the order's date"* is a date, not a Date class
3. **VERBS** between nouns → candidate associations and operations
4. Apply the tests: **is-a?** → triangle. **dies-with?** → filled diamond. **borrowed reference?** → dashed. Then add multiplicities from words like each, one, many, up to.

THE SENTENCE

A **customer** *places* an **order** for one or more **products**; each **order line** *refers to* exactly one **product**.

THE PARSE




Not every noun becomes a class, **the system itself, the UI, and "date" usually don't.** Test: state + behavior + identity + does the system care?

# Step 1: read, circle, decide

REQUIREMENTS

*Members register at the library and may borrow up to 5 copies at a time. The library owns several copies of each book. Each copy belongs to exactly one book. Each book lists one or more authors. Librarians check copies in and out and can look up which member currently holds a copy.*

CANDIDATE CLASSES

Member | Copy | Book | Author | Librarian | ~~library~~ *the system itself*

VERBS

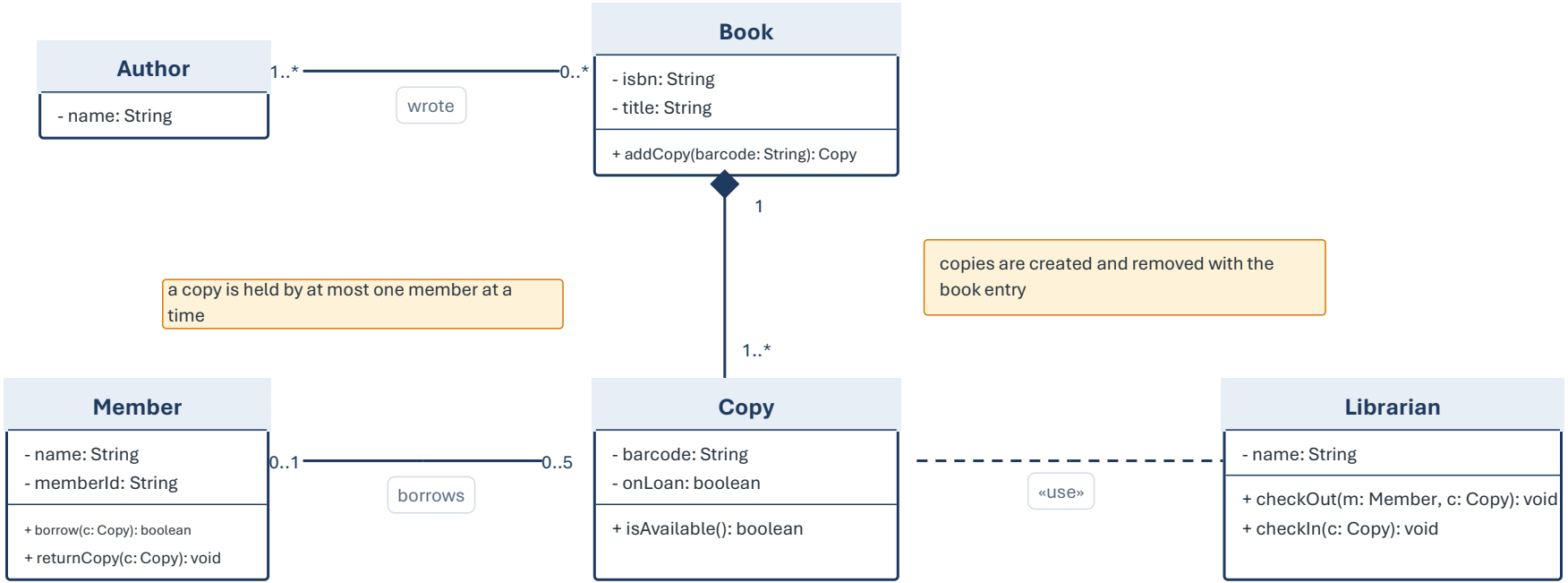
registers | borrows | owns | belongs to | lists | checks in/out

A ‘up to 5’ → a bounded multiplicity 0..5 on Member, Copy

B ‘each copy belongs to exactly one book’ → composition: **Book ◆— copies**, mult 1 near Book

C ‘librarian checks copies in and out’ → operations on **Librarian** (checkIn, checkOut), plus a dependency arrow, not an association

# Step 2: the model




5 classes · 4 relationship kinds · every multiplicity decidable from the requirements text, that is a complete structural model.

# The diagram is the code

One Java file per class, here are three of the five.

```
public class Book {
private String isbn;
private String title;

private List<Copy> copies = new ArrayList<>();   // ◆ composition

public Copy addCopy(String barcode) {
Copy c = new Copy(barcode);     // copies are BORN inside Book
copies.add(c);
return c;
}
}

public class Copy {
private String barcode;
private boolean onLoan;
private Member heldBy;          // 0..1 association — may be null
}

public class Member {
private String name;
private List<Copy> borrowed = new ArrayList<>();  // 0..5 association

public boolean borrow(Copy c) {
if (borrowed.size() >= 5) return false;   // 'up to 5', enforced
borrowed.add(c);
c.heldBy = this; c.onLoan = true;       // update the other end
return true;
}
}
```

**◆ composition**

object creation INSIDE the whole, the whole mints its parts

**0..1 association**

a nullable single reference: heldBy is set and cleared by the borrow / return logic

**0..5 association**

an if-statement guards the list, bounded multiplicity is executable design

# Same model, different language: Python

```
class Copy:
def __init__(self, barcode: str):
self.barcode = barcode
self.on_loan = False
self.held_by: "Member | None" = None     # 0..1 association

class Book:
def __init__(self, isbn: str, title: str):
self.isbn = isbn
self.title = title
self._copies: list[Copy] = []          # ◆ composition

def add_copy(self, barcode: str) -> Copy:
copy = Copy(barcode)                   # born inside the whole
self._copies.append(copy)
return copy

class Member:
MAX_COPIES = 5                         # 0..5, enforced

def __init__(self, name: str):
self.name = name
self.borrowed: list[Copy] = []

def borrow(self, copy: Copy) -> bool:
if len(self.borrowed) >= self.MAX_COPIES:
            return False
self.borrowed.append(copy)
        return True
```

**visibility**

becomes convention: _copies says internal. Python trusts you __name triggers name-mangling, still not a wall

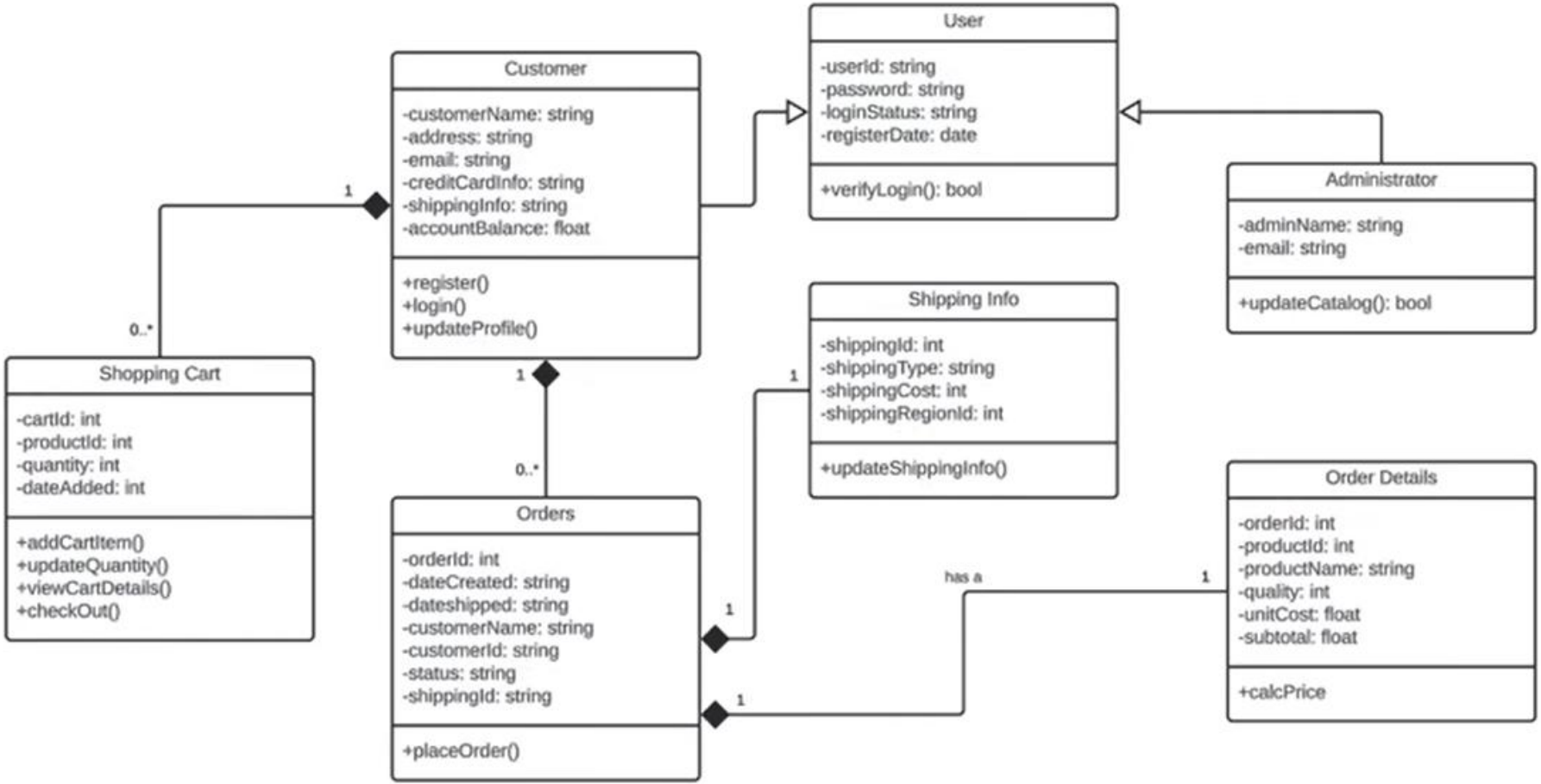
**same choices**

same model, same multiplicities: 0..5 is still an if, composition is still creation inside the whole

**the point**

the DIAGRAM is the design; Java and Python are two spellings of it. Interfaces in Python: ABC or typing.Protocol (3.8+).

UML is language-independent: that is the whole point of a modeling language.


User
-userId: string
-password: string
-loginStatus: string
-registerDate: date
+verifyLogin(): bool
Customer
-customerName: string
-address: string
-email: string
-creditCardInfo: string
-shippingInfo: string
-accountBalance: float
+register()
+login()
+updateProfile()
Administrator
-adminName: string
-email: string
+updateCatalog(): bool
Shopping Cart
-cartId: int
-productId: int
-quantity: int
-dateAdded: int
+addCartItem()
+updateQuantity()
+viewCartDetails()
+checkOut()
Shipping Info
-shippingId: int
-shippingType: string
-shippingCost: int
-shippingRegionId: int
+updateShippingInfo()
Orders
-orderId: int
-dateCreated: string
-dateshipped: string
-customerName: string
-customerId: string
-status: string
-shippingId: string
+placeOrder()
Order Details
-orderId: int
-productId: int
-productName: string
-quality: int
-unitCost: float
-subtotal: float
+calcPrice
1
0..*
1
0..*
1
1
1
1
has a
1

**1 Multiplicity on the wrong end**

1 near Student read as “one student, one course”

**FIX →** the number near class X counts X’s, say the sentence out loud first

**2 Inheritance for has-a**

class Car extends Engine

**FIX →** only true is-a gets a triangle; has-a is a line or a diamond

**3 The God class**

one class, forty attributes, every operation

**FIX →** split by responsibility, each class earns its name in one sentence

**4 Modeling the UI / the system**

System, Screen and Button classes in a domain model

**FIX →** the domain model describes the problem world, not the software around it

**5 Attribute or class?**

Address: a String field or its own class?

**FIX →** own structure + behavior → class; just a label → attribute

**6 Diamond overkill**

hollow and filled diamonds on every line

**FIX →** when unsure: plain association + a note, the distinction earns little in practice (Fowler, UML Distilled)

# Pick one tool, learn it once

- **PlantUML:** diagrams as text; diffs and versions with your code
- **Mermaid:** same idea; renders inside GitHub & GitLab markdown
- **diagrams.net (draw.io):** free drag-and-drop, no account, runs in the browser
- **Lucidchart:** polished, team-friendly, templates
- **StarUML · Visual Paradigm:** full OOAD suites

PlantUML text

```
@startuml
class Order {
-total: Money
+place(): void
}
class OrderLine { -qty: int }
class Customer { -name: String }

Customer "1" -- "0..*" Order : places
Order "1" *-- "1..*" OrderLine : contains
@enduml
```

Six lines of text → a correct diagram. Multiplicities are quoted end-first: “1” on the Customer side, “0..*” on the Order side. The * before -- marks composition.

For this course: **PlantUML or diagrams.net:** both free, both enough.

# Model the Coffee Corner

REQUIREMENTS

*Customers place orders. An order contains one or more drinks; every drink is a Coffee or a Tea, in size small, medium, or large. Customers may hold a loyalty card; each card collects stamps, ten stamps earn a free drink. Baristas prepare the drinks of each order.*

1. Circle the nouns → expect 4–6 classes: Customer, Order, Drink (→ Coffee / Tea?), LoyaltyCard, Barista…

2. Apply the tests: **Order–Drink:** ◆ or plain line? **Coffee/Tea–Drink:** which arrow? **Customer–LoyaltyCard:** which multiplicity? (hint: 0..1)

3. Draw the full diagram with multiplicities. **One photo per pair.**

Stuck? Say sentences out loud: **“an order CONTAINS drinks”.** And sizes? They are one attribute with three values: not a class.